

Treinamento de GPT com base nos textos de Machado de Assis

231003406 - Lucas Gabriel de Oliveira Lima

Abstract—O projeto é baseado na construção de um Generative Pre-trained Transformer treinado com todos os textos de Machado de Assis disponíveis publicamente. A ideia é aplicar os conceitos de Transformer e Multi-head Self Attention de forma similar a como foi feita por Andrej Karpathy em um projeto similar, mas que difere na escolha do autor para fazer o treinamento do modelo.

Index Terms—LLM, Transformer, Attention, Machado de Assis, Treinamento

I. INTRODUÇÃO

ESTE trabalho apresenta a implementação de um modelo de linguagem generativo inspirado na arquitetura GPT e treinado exclusivamente com textos de Machado de Assis. O principal objetivo é aplicar, de forma prática, conceitos como embeddings, atenção causal com múltiplas cabeças, blocos Transformer, conexões residuais, normalização em camadas e geração autoregressiva. A implementação tem como referência didática o projeto de construção de um GPT apresentado por Andrej Karpathy, adaptando-o para um corpus em língua portuguesa formado pela obra do autor brasileiro.

O modelo foi desenvolvido em Python com a biblioteca PyTorch e utiliza tokenização em nível de caractere. O corpus foi dividido em conjuntos de treinamento e validação, possibilitando acompanhar a evolução da função de perda e avaliar a capacidade de generalização da arquitetura. Também foram implementadas estratégias de regularização e otimização, como dropout, AdamW, aquecimento linear da taxa de aprendizado, decaimento cossenoidal e imitação da norma dos gradientes.

Após o treinamento, o desempenho do modelo foi analisado quantitativamente por meio das perdas de treinamento e validação e, qualitativamente, pela geração de textos condicionados por trechos conhecidos de Machado de Assis. Essa avaliação permite observar tanto a capacidade do modelo de reproduzir padrões ortográficos, sintáticos e estilísticos do corpus quanto suas limitações na manutenção da coerência semântica em sequências mais longas.

A implementação completa, incluindo a preparação dos dados, o treinamento do modelo e os procedimentos de geração de texto, encontra-se disponível no repositório <https://github.com/lucasdbr05/gpt-axe/>.

Agosto 28, 2026

II. PREPARAÇÃO DOS DADOS

A primeira etapa para a preparação do modelo foi agregar toda a obra de Machado de Assis em um arquivo .txt de forma similar a como foi feito no vídeo de referência de Andrej Karpathy.

O corpus utilizado no treinamento do modelo foi construído a partir de um acervo de textos de Machado de Assis disponibilizado em formato compactado. O processo de preparação é executado pelo script `scripts/machado.py`, responsável por extrair, validar, padronizar e reunir as obras em um único arquivo textual.

Inicialmente, o script obtém o arquivo compactado do acervo ou utiliza um arquivo zip fornecido localmente. Em seguida, são selecionados apenas os textos organizados nas categorias romance, conto, poesia, crônica, teatro, crítica, tradução e miscelânea.

Cada arquivo é lido com codificação UTF-8, a marca de ordem de bytes é removida quando presente e padronizando-se as quebras de linha e seus conteúdos são concatenados no arquivo `data/machado_de_assisobra_completa.txt`.

III. ARQUITETURA

A arquitetura final implementada segue o paradigma de modelos de linguagem autoregressivos baseados em Transformers do tipo decoder-only, semelhante à família GPT. O objetivo do modelo é estimar a probabilidade do próximo token a partir de uma sequência de tokens anteriores, utilizando atenção causal para impedir o acesso a posições futuras durante o treinamento.

Na versão final do modelo, utilizamos os seguintes hiperparâmetros:

block_size = 512
n_embd = 512
n_head = 8
n_layer = 6
dropout = 0.1

Sendo que eu considero o seguinte:

block_size: tamanho máximo da sequência de contexto processada pelo modelo, em número de tokens

n_embd: dimensão vetorial usada para representar cada token

n_head: número de cabeças de atenção

n_layer: número de blocos Transformer empilhados

dropout: probabilidade de desativar ativações durante o treinamento para regularização

A entrada do modelo é composta por sequências de até 512 tokens. Cada token é inicialmente convertido em uma representação vetorial de 512 dimensões por meio de uma camada de token embedding. Em paralelo, é utilizada uma tabela de embeddings posicionais aprendidos, responsável por incorporar informação sobre a posição de cada token dentro da sequência. As duas representações são somadas antes de serem encaminhadas às camadas Transformer.

A fim tornar a implementação mais modular, quebrei a arquitetura em diferentes camadas:

A. MultiHead Self-Attention

A classe de **CausalMultiHeadSelfAttention**, responsável por implementar o conceito de MultiHead Self-Attention, é responsável pela projeção de cada representação de entrada em vetores de query, key e value, distribui esses vetores entre múltiplas cabeças de atenção e aplica a atenção por meio de produto escalar escalonado sob uma restrição causal. Isso garante que cada posição possa atender apenas a si mesma e às posições anteriores. As saídas de todas as cabeças de atenção são, em seguida, concatenadas e projetadas linearmente de volta para a dimensão de embedding do modelo.

A classe é inicializada com os valores da dimensão vetorial dos tokens (`n_embd`) e do número de cabeças de atenção (`n_head`). Inicialmente, é feita uma divisão por cabeça, então cada cabeça irá operar com n_embd / n_head dimensões (no nosso caso, 64 dimensões), e as cabeças passarão a operar paralelamente.

As projeções de query, key e value são, portanto, reorganizadas de forma que cada cabeça execute independentemente uma operação de scaled dot-product attention.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Embora todas processem a mesma sequência, cada cabeça pode aprender padrões diferentes de dependência entre os tokens, como relações locais, estruturas sintáticas ou dependências de maior alcance. Ao final, as saídas das oito cabeças são concatenadas, reconstruindo uma representação de 512 dimensões, que é posteriormente projetada para a dimensão original do modelo por uma camada linear.

Em uma das aulas que assisti de transformers na preparação para esse projeto, uma das coisas que me fez entender melhor o funcionamento, foi entender que, em termos práticos, essa operação diz o seguinte:

Q: o que estou procurando

K: como posso ser encontrado

V: o que tenho para fornecer

Só que esses conceitos não são definidos manualmente, o modelo aprende automaticamente durante o treinamento quais características devem ser usadas para busca e transporte de informação.

Após isso tudo, é aplicado um dropout, que é uma técnica de regularização que tem como função principal reduzir o overfitting durante o treinamento. Como o dropout tem valor de 0.1, essa porcentagem de ativações serão descartadas aleatoriamente durante o treinamento.

B. Feed Forward

Feed Forward é a etapa posterior ao Multihead Attention. Ela é um MultiLayer Perceptron (MLP) responsável por fazer um processamento individual da informação contextual adquirida por cada token.

Na arquitetura, para cada representação, primeiramente é feito uma expansão (se cada token é representado por vetores

de tamanho 512, passa a ser representado por um de tamanho 2048). A ideia dessa mudança é permitir o aprendizado de informações mais complexas.

Então, é aplicada a função de ativação GELU, que introduz não linearidade.

Por fim, o vetor retorna a sua dimensão original e então é aplicado mais um dropout no final.

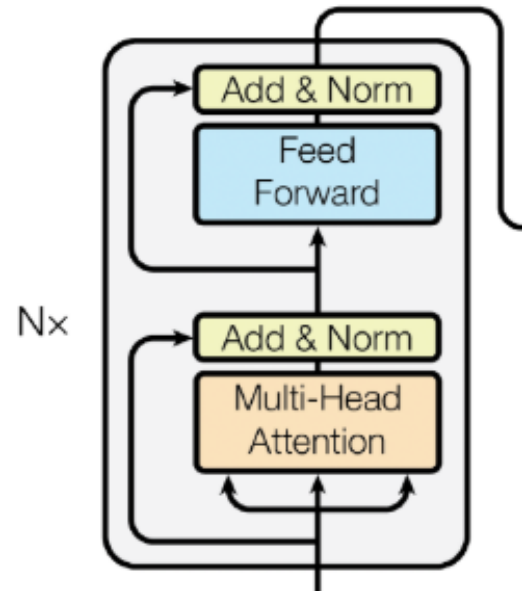


Fig. 1. Transformer Block

C. Transformer Block

Essa classe não introduz a implementação de uma abstração totalmente nova, ela apenas deixa mais modular o fluxo de MultiHead Attention e Feed Forward.

Optei por implementar assim para deixar a implementação mais próxima do design da seguinte composição proposta como na Figura 1, pelo artigo "Attention is All You Need".

D. GPT Language Model

Após a construção dos módulos anteriores, a classe **GPT-LanguageModel** organiza a arquitetura em um modelo de linguagem completo. Ela organiza os embeddings, empilha os blocos Transformer, produz os logits, calcula a loss e faz geração autoregressiva.

Internamente, ela gerencia uma tabela de embedding dos tokens.

A tabela de embeddings de token de tamanho `n_embd` para cada token, sendo cada token uma letra do vocabulário do corpus, a qual tem a seguinte dimensão: `vocab_size = 143`

O vocabulário foi definido por um script, sendo no total o seguinte:

TABLE I
VOCABULÁRIO DE CARACTERES UTILIZADO PELO MODELO

␣	\n	!	"	\$	%	&	'	(	)	*	+
,	-	.	/	0	1	2	3	4	5	6	7
8	9	:	;	=	?	A	B	C	D	E	F
G	H	I	J	K	L	M	N	O	P	Q	R
S	T	U	V	W	X	Y	Z	_	'	a	b
c	d	e	f	g	h	i	j	k	l	m	n
o	p	q	r	s	t	u	v	w	x	y	z
ı	§	ª	«	º	´	°	»	½	¿	À	Á
Â	Ã	Ç	È	É	Ê	Ë	Í	Ó	Ô	Õ	Ú
Û	Ü	à	á	â	ã	ä	ç	è	é	ê	ë
ì	í	î	ï	ñ	ò	ó	ô	õ	ö	ù	ú
û	ü	œ	–	—	—	‘	’	“	”	...	

Já a tabela de embeddings de posição representa a posição de cada token numa sequência de tamanho **block_size**, com cada embedding tendo dimensão de tamanho **n_embd** também.

Além disso, a classe gerencia uma pilha de blocos de Transformer, de tamanho **n_layer**. Cada bloco recebe a representação anterior e a refina, incorporando contexto por meio de atenção e processamento pela MLP.

Depois de todos os blocos de Trasnformer, uma normalização final (LayerNorm final) é aplicada antes da projeção para o vocabulário. Isso é compatível com a estrutura Pre-LayerNorm usada no restante da arquitetura.

No final do processo, há uma camada responsável por transformar a representação final de cada posição em um vetor do tamanho do vocabulário. Assim, a representação passa de tamanho **n_embd** para o tamanho de **vocab_size**. Os valores dessa representação final são o que nós chamamos de **logits**, que representam a preferência do modelo por cada possível token seguinte.

1) *Forward em treinamento*: O método `forward` define como uma sequência de entrada é processada pelo modelo durante o treinamento e durante a inferência.

O foward inicia com um tensor `dinput` no formato (B, T) , sendo B o número de sequências no batch, e T o número de tokens por sequência (`block_size`).

O método inicialmente converte cada identificador de token em uma representação vetorial aprendida por meio da tabela de embeddings de tokens. além disso, é obtido um embedding posicional correspondente a cada posição da sequência.

Essas duas representações são somadas, formando a representação inicial de entrada:

$$x_t = E_{token}(x_t) + E_{position}(t),$$

onde E_{token} representa o embedding aprendido do token localizado na posição t , enquanto $E_{position}$ representa o embedding aprendido associado àquela posição.

O resultado,então, passa a ter o shape (B, T, n_embd) .

O esse tensor é passado para a sequência de blocos de Transformer.Esses blocos transformam as representações iniciais em representações contextualizadas, de forma que cada posição passe a incorporar informações dos tokens anteriores.

Após o último bloco Transformer, é aplicada uma operação final de Layer Normalization (H_L representa a saída do último bloco Transformer):

$$H = LayerNorm(H_L),$$

Em seguida, as representações normalizadas são projetadas para a dimensão do vocabulário por meio da camada de saída do modelo, gerando então os **logits** produzidos pelo modelo. O shape do tensor passa a ser $(B, T, vocab_size)$.

Durante o treinamento, os tokens esperados são fornecidos por meio do argumento `targets`. Como a função de entropia cruzada trata cada previsão como um exemplo de classificação independente, os logits são reorganizados de $(B, T, vocab_size)$ para $(B \cdot T, vocab_size)$, enquanto os targets são reorganizados de (B, T) para $(B \cdot T)$. A função de perda é então calculada por meio da entropia cruzada, que implementa internamente o seguinte:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i),$$

onde $N = B \cdot T$ e $P(y_i)$ representa a probabilidade atribuída pelo modelo ao token correto na posição i .

Assim, o método `forward` implementa a transformação completa de uma sequência de identificadores de caracteres em previsões para o próximo caractere em cada posição da sequência.

2) *Geração Autoregressiva (generate)*: O método `generate` utiliza os logits produzidos pelo modelo para gerar novos tokens de forma autoregressiva. O processo de geração utiliza apenas a previsão correspondente à última posição do contexto atual para escolher o próximo token.

A cada iteração, o contexto é limitado aos últimos `block_size` tokens, garantindo que o comprimento da sequência fornecida ao modelo não ultrapasse o tamanho máximo de contexto utilizado durante o treinamento.

Em seguida, o contexto condicionado é processado pelo método `forward`, que produz um tensor de logits com formato $(B, T, vocab_size)$

Como a geração necessita apenas da previsão para o próximo token, são selecionados os logits correspondentes à última posição da sequência.

IV. PERDA ESTIMADA

Ao longo do treinamento, utilizamos uma função de calculo de perda estimada para medir o desempenho do modelo ao longo do processo. O modelo é otimizado utilizando a função de cross-entropy loss, que mede o logaritmo negativo da probabilidade atribuída ao próximo token correto. A perda final é calculada como a média das previsões de todos os tokens nos lotes avaliados. Valores menores indicam que o modelo atribui maior probabilidade aos próximos caracteres esperados.

V. TAXA DE APRENDIZADO

Utiliza-se um escalonador de taxa de aprendizado com aquecimento (warmup) linear inicial, seguido de decaimento

cossenoidal. Durante as primeiras 100 iterações, a taxa de aprendizado aumenta linearmente até o valor máximo. Após a fase de aquecimento, ela diminui suavemente segundo um cronograma cossenoidal até atingir um décimo da taxa de aprendizado inicial.

Essa estratégia ajuda a evitar atualizações instáveis no início do treinamento.

VI. RESULTADOS

A implementação completa do modelo, incluindo os scripts de preparação dos dados, o código de treinamento e os procedimentos de geração de texto, está disponível publicamente no repositório GitHub: <https://github.com/lucasdb05/gpt-axe>

A. Versão inicial

Para avaliar o comportamento da arquitetura em uma configuração reduzida, foi treinado um modelo com `batch_size = 32`, contexto máximo de 64 tokens, dimensão de embedding igual a 128, 4 cabeças de atenção e 3 blocos Transformer. O treinamento foi executado por 12500 iterações, utilizando taxa de aprendizado de 3×10^{-4} , `dropout = 0.1` e 100 iterações para avaliação. Essa configuração resulta em um modelo significativamente menor do que as versões com maior dimensão de embedding e maior profundidade, permitindo observar o desempenho de uma arquitetura mais compacta.

A primeira versão, utilizando a mesma arquitetura explicada, foi utilizando os seguintes hiperparâmetros:

```
block_size = 64
n_embd = 128
n_head = 4
n_layer = 3
dropout = 0.1
```

Ao final, do treinamento, o modelo atingiu um loss de **1.6003**.

Esse valor indica que o modelo conseguiu aprender padrões estatísticos relevantes do corpus, embora ainda apresente limitações na geração de sequências totalmente coerentes.

Analisando uma geração autoregressiva de 500 novos tokens, uma saída produzida foi:

"uma senhora, he levantas de caridade, de convidar uma maridade, nem na política, perguntava uma desespera e me exiquite, constituição da flor cova outras e que modesta; por um tempo ao poeta io da conceles de grande do tibatre a dos considados de Saíes de Aleveredadeiros, com aqui eu visto para de casar o mesmo tempo mal e rompreendo portas aos nomes do coração de clonistro; o trocupo é de pouco ao arma. Cercosas, as repensa do sanse esse talvez estando como por estou o seu pareceu dua. Felito no"

B. Versão Final

A versão final foi treinada com uma arquitetura maior, composta por seis blocos Transformer, oito cabeças de atenção e embeddings de 512 dimensões. O contexto máximo foi

ampliado para 512 caracteres, permitindo que o modelo considerasse dependências mais longas durante a previsão do próximo token.

A implementação final foi executada em um servidor com sistema operacional Ubuntu, equipado com uma GPU NVIDIA GeForce RTX 5080 e 64 GB de memória RAM. O treinamento completo, considerando as 12 500 iterações definidas para essa configuração, teve duração aproximada de uma hora e quarenta minutos. O uso da GPU permitiu acelerar as operações matriciais e o processamento paralelo característicos dos blocos Transformer, tornando viável o treinamento da arquitetura final com maior dimensão de embeddings, mais camadas e um contexto de 512 caracteres.

O modelo final possui **19.238.400 parâmetros treináveis** e utiliza um vocabulário de **143 caracteres**. O corpus contém 10.977.812 caracteres, dos quais 9.880.030 foram destinados ao treinamento e 1.097.782 à validação, correspondendo a uma divisão de aproximadamente 90% e 10%, respectivamente.

Os principais parâmetros utilizados no treinamento foram:

```
batch_size = 32
block_size = 512
n_embd = 512
n_head = 8
n_layer = 6
dropout = 0.1
max_iters = 12500
learning_rate = 10-3
```

O treinamento foi realizado utilizando o otimizador AdamW, com coeficientes $\beta_1 = 0.9$ e $\beta_2 = 0.95$, weight decay igual a 0.1 e limitação da norma dos gradientes em 1.0. A Tabela II apresenta alguns pontos representativos da evolução da perda. Cada valor foi estimado pela média de 100 lotes.

TABLE II
EVOLUÇÃO DA PERDA DA VERSÃO FINAL

Iteração	Treinamento	Validação
0	5.1206	5.1130
1 000	1.4142	1.5846
3 000	1.1774	1.4027
5 000	1.0950	1.3563
7 000	1.0284	1.3389
8 900	0.9743	1.3266
10 000	0.9490	1.3327
12 400	0.9110	1.3441

Ao término do treinamento, também foi registrada uma perda de **0.9495** no último minibatch processado. Esse valor não deve ser confundido com a perda média apresentada na tabela, pois corresponde somente a um lote específico e, portanto, está sujeito a maior variação estatística.

De modo geral, a versão final apresentou uma evolução significativa em relação à configuração inicial. O aumento da capacidade do modelo e do tamanho do contexto resultou em menor perda e em textos mais estruturados. Os resultados confirmam que uma arquitetura Transformer decoder-only, mesmo

treinada em nível de caractere e com aproximadamente 19 milhões de parâmetros, é capaz de aprender aspectos relevantes da escrita presente no corpus de Machado de Assis. Entretanto, a diferença entre as perdas de treinamento e validação mostra que técnicas como early stopping, aumento do corpus ou regularização adicional poderiam melhorar a capacidade de generalização do modelo.

1) *Geração autoregressiva inicial*: Analisando uma geração autoregressiva de 500 novos tokens, uma saída produzida foi:

"mestre do conteato que vinha tocar no horizonte, o radil de Lacerda extremamente lânguida, como a verdadeira razão da forma principal, e antes de tudo isso me era preciso acabar com o desejo de ver o resguardo das graças do papel, que se esqueceram da inteligência moral. Entendi o que era a câmara boa principal; depois vi que, em consciência, a desforra da reclamação, da parte dele na própria câmara, foram as próprias obras do país, e na proposta da fatal notícia. Tal foi a imprensa com mais côm"

Além disso, fiz um teste iniciando o contexto da decodificação com alguns dos trechos dos escritos de Machado de Assis que ficaram mais famosos entre seus leitores, alguns dos resultados que obtive foram os seguintes:

2) *Primeiro prompt*: "**Capitu era Capitu, isto é, uma criatura mui particular** que os dois primeiros, observavam outra coisa ao coração, e bastavam-lhe as mesmas ordens. Quando acabou; ouviu matar alguns grandes rápidos ou pessoas, tosseguiram pelo fim da casa. A semana passou as janelas para o paço; nas ocasiões foi o padre com que ele casou com a filha. Capitu aposentou-se de fora daquela casa mais franca que nem sempre. Conservou-se o pai à aguiar e pediu-lhe que desfazer as medias. Um dia mais antes, jurei a concluir tudo o que podia. Os carros eram ainda como um só dia, além de minha casa, como se carecia alguns em semelhante coisa. Não lho poderia anter alcançar a mobília do rimo econômico. Notemos que esta moça tinha as suas forças, a ambição e a semelhança de "

3) *Segundo prompt*: "**Olhos de ressacar**. A principal carta foi comparada e inesperada, e não permitiu o desenlace no passado, que o seu reparo fez esquecer o mais nobre amor da mesma alma. A segunda, porém, é que o serviço foi a da concorrência do casamento. Não se desconhece no primeiro passo? O plano arrebatado era pouco mais escasso e mais seguro; a natureza deste houve idéia aflitiva e clara, que era pouca e igualmente inconsistente. A verdade é que a força de meninos não era a natureza; mas, sim, era a de algum modo que a mãe percebeu, e eu, tendo tido a repetir-lhes agora toda a estatura do sacristão, ficaria sendo pressentido repentinamente curto. O sacristão não a sabia. Contava esses dois motivos que D. Benedita, como v "

4) *Terceiro prompt*: "**Não tive filhos. Não transmiti a nenhuma criatura o legado da nossa miséria** antiga; e principalmente quando os manuscritos me levantaram para casa. Os maus eram os cumprimentos e desapareceram juntos. Não admirava que a ligação de Maria Benedita fosse a notícia do caráter de Rubião, em suma, antes da morte, e da integridade da noite de Vicente, indo a ele e a arranjar caminho, não lhe confiou palavras de modo afirmativo, apesar de ajuda e de

pedir a notícia. Não obstante os carinhos de feitor magros, iam logo ao presente carro da daminha, e davam ao presente para a visita dos irmãos para cada uma, até que Rubião tocava em suas mãos. Com efeito, Sofia acabou com o marido. Entre os dois presidentes, as duas estantes e os sentimentos que compareceram, e sentiam falar a"

5) *Quartao prompt*: "**Ao verme que primeiro roeu as frias carnes do meu cadáver dedico como saudosa lembrança estas memórias póstumas** novas da esposa. Um compadre que o sol e o sol de tabuleta nos trazemos esfalfada e dentro de fazer bem as rosas de Rosa, Estrelas de Ataíde com elas, mas são as suas mágoas precisas a esta grande perícia. Sofre essas consolações entusiásticas, as relíquias do canto; as sensualidades devem chamar a meia devassa e o espírito calcula. Não se deve calcular a soma do espaço; e a perspicácia de um dos vícios da comédia seriam tão fracas. Até lá ficaria livre e alcançado, em detrimento, para a sociedade de Comon e ao mal estipulado. O tempo teimoso parece convencer que esta grande forma primitiva nesse temor, e a mais imunda arte pode ser a melhor competente e grave prosa da mocidade e do present"

VII. EXTENSÃO PARA PERGUNTAS E RESPOSTAS

Como extensão deste trabalho, o modelo poderia ser adaptado para responder a questões relacionadas ao universo literário de Machado de Assis, incluindo perguntas sobre personagens, acontecimentos, obras, temas e características estilísticas. Para isso, seria necessário realizar um ajuste supervisionado com pares de perguntas e respostas, ensinando o modelo a produzir respostas diretas em vez de apenas continuar uma sequência textual.

Essa abordagem poderia ser combinada com uma arquitetura de **Retrieval-Augmented Generation (RAG)**, na qual as obras seriam divididas em trechos e armazenadas em um índice de busca. Diante de uma pergunta, o sistema recuperaria os fragmentos mais relevantes e os forneceria como contexto para a geração da resposta. Além de reduzir respostas sem fundamentação no corpus, esse mecanismo permitiria indicar a obra ou o trecho utilizado como fonte. Para viabilizar essa aplicação, também seria recomendável ampliar o tamanho máximo de contexto, utilizar tokenização por subpalavras e avaliar o sistema quanto à correção factual, fidelidade às obras e capacidade de reconhecer perguntas que não podem ser respondidas com os dados disponíveis.

Para esse objetivo, o RAG é especialmente importante, mas seria pouco realista esperar que um modelo de 19 milhões de parâmetros memorizasse com precisão todas as obras e, ao mesmo tempo, respondesse factualmente sem consultar o corpus.

VIII. CONCLUSÃO

Este trabalho demonstrou, na prática, a construção e o treinamento de um modelo de linguagem baseado na arquitetura Transformer e treinado com a obra de Machado de Assis. A versão final, com aproximadamente 19 milhões de parâmetros, apresentou redução significativa das perdas de treinamento e validação em relação à configuração inicial, além de produzir

textos que reproduzem características ortográficas, estruturais e estilísticas presentes no corpus.

Embora as gerações ainda apresentem limitações de coerência, concordância e continuidade semântica, os resultados confirmam que uma arquitetura decoder-only com tokenização em nível de caractere é capaz de aprender padrões relevantes da língua portuguesa e do estilo do autor. Como trabalhos futuros, poderiam ser explorados o uso de tokenização por subpalavras, conjuntos de dados mais amplos, seleção do melhor modelo por validação, técnicas adicionais de regularização e arquiteturas de maior capacidade, buscando melhorar a generalização e a consistência dos textos gerados.

IX. REFERÊNCIAS

REFERÊNCIAS

- [1] A. Karpathy, “nanoGPT,” GitHub. Disponível em: <https://github.com/karpathy/nanoGPT>. Acesso em: 28 set. 2026.
- [2] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017. PDF disponível em: <https://arxiv.org/pdf/1706.03762>.
- [3] A. Karpathy, “Let’s build GPT: from scratch, in code, spelled out,” YouTube. Disponível em: <https://www.youtube.com/watch?v=kCc8FmEb1nY&list=PLAqIrjKxbuWI23v9cThsA9GvCAUhRvKZ&index=10>. Acesso em: 28 set. 2026.
- [4] Free Code Camp, “Create a Large Language Model from Scratch with Python – Tutorial” YouTube. Disponível em: <https://www.youtube.com/watch?v=UU1WVnMk4E8&t=944s>. Acesso em: 28 set. 2026.
- [5] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017. Disponível em: <https://arxiv.org/abs/1706.03762>.
- [6] T. B. Brown et al., “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems*, vol. 33, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>.